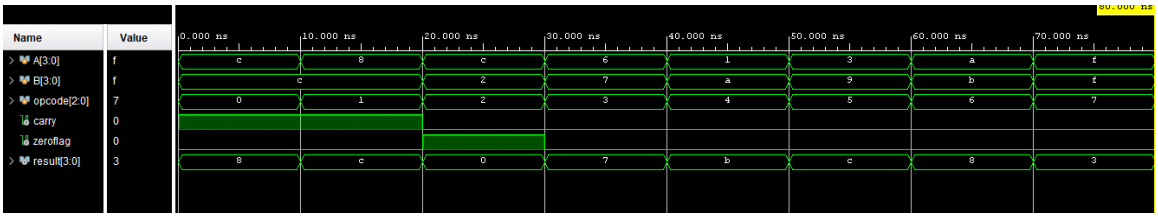
```
//==============================================================
                    // HDL-VERILOG Scripts
//================================================================
// Objective Designing 4 bit input Arithmetic;
// Combinational Logic|Targetted operation=8;
// 4 Bit -ALU Taken here
//=================================================
// Verilog Script is used here
=======================================================
// MODULE Design
//=======================================================
//Part 1
module ALU_4Bit(
//Part 2
        input wire [3:0] A,
        input wire [3:0] B,
        input wire [2:0] opcode,
        output wire zeroflag,
        output reg carry,
        output reg [3:0] result
        );
////////////////////
assign zeroflag=(result==4'b0000);
//Part 3--COM BIN NE
always@(*)begin
```

```
  carry=0; //default

case(opcode)

3'b000:{carry,result}=A+B;

3'b001:{carry,result}=A-B;

3'b010:result=A&B;

3'b011:result=A|B;

3'b100:result=A^B;

3'b101:result=~A;

3'b110:result=A<<2;

3'b111:result=A>>2;

endcase

end

endmodule
//==============================================================================
//==============================================================================

// Test bench for ALU----Verilog used here

//==============================================================================

//Note: Unsigned decimal Considered here; Operand A is taken for shift and NOT operation

//=========================================================================

module tb_ALU_4Bit();

reg [3:0]A;

reg [3:0]B;

reg [2:0]opcode;

wire carry;

wire zeroflag;

wire [3:0]result;
```

```
ALU_4Bit uut(
        .A(A),
        .B(B),
        .opcode(opcode),
        .result(result),
        .carry(carry),
        .zeroflag(zeroflag)
        );
initial begin
////////////////////////////////////////////// Test1
A=4'd12;
B=4'd12;
opcode=3'b000;
#10;
$display( "operation result=%b carry=%b zeroflag=%b",result,carry,zeroflag);
//////////////////////////////////////////////Test2
A=4'd8;
B=4'd12;
opcode=3'b001;
#10;
$display( "operation result=%b carry=%b zeroflag=%b",result,carry,zeroflag);
//////////////////////////////////////////////////Test3
A=4'd12;
B=4'd2;
```

```
opcode=3'b010;

#10;

$display( "operation result=%b carry=%b zeroflag=%b",result,carry,zeroflag);

//////////////////////////////////////////////////Test4

A=4'd6;

B=4'd7;

opcode=3'b011;

#10;

$display( "operation result=%b carry=%b zeroflag=%b",result,carry,zeroflag);

//////////////////////////////////////////////////////Test5

A=4'd1;

B=4'd10;

opcode=3'b100;

#10;

$display( "operation result=%b carry=%b zeroflag=%b",result,carry,zeroflag);

//////////////////////////////////////////////////////////Test6

A=4'd3;

B=4'd9;

opcode=3'b101;

#10;

$display( "operation result=%b carry=%b zeroflag=%b",result,carry,zeroflag);

//////////////////////////////////////////////////////////////Test7

A=4'd10;

B=4'd11;

opcode=3'b110;
```

```
#10;

$display( "operation result=%b carry=%b zeroflag=%b",result,carry,zeroflag);

//////////////////////////////////////////////////////////////Test8

A=4'd15;

B=4'd15;

opcode=3'b111;

#10;

$display( "operation result=%b carry=%b zeroflag=%b",result,carry,zeroflag);

$finish;

end

endmodule

///==============================================================================
/// ANALYSIS

# run 1000ns

operation result=1000 carry=1 zeroflag=0

operation result=1100 carry=1 zeroflag=0

operation result=0000 carry=0 zeroflag=1

operation result=0111 carry=0 zeroflag=0

operation result=1011 carry=0 zeroflag=0

operation result=1100 carry=0 zeroflag=0

operation result=1000 carry=0 zeroflag=0

operation result=0011 carry=0 zeroflag=0
```